Failure (force the parser to actually fail) — Hints

Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — Forcing a real failure: "check after"
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — Forcing a real failure: "check after"

Basic Version

The previous exercise showed that "shape during" (`.with_structured_output`) almost never fails — the model is forced into the shape. To see a real parser failure, use the other method from the Output parsers topic: **check after**.

The model writes free text, and a `PydanticOutputParser` at the end of the chain checks that text afterwards. If the prompt never tells the model the shape, the model answers in a sentence — and the parser fails.

- `parser = PydanticOutputParser(pydantic_object=Rating)` with `class Rating(BaseModel): score: int`
- `chain = prompt | model | parser`, with the prompt `"Rate {thing} from 1 to 10."`
- Run it inside `try/except Exception as e:` and print `type(e).__name__`.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

The error is `OutputParserException` (from `langchain_core.exceptions`) — catch it by name, never a bare `except`, so a network error or a bad key still fails loudly.

Then fix the cause, not just the symptom: tell the model the shape. `parser.get_format_instructions()` returns that text, and `.partial(format_instructions=...)` fills a `{format_instructions}` slot once, so callers still pass only `{thing}` (the Prompt templates topic).

`parser.parse("some text")` checks any text with no API call — a free way to test the failure again.

Difference between Basic and Intermediate: Basic finds the exact error class. Intermediate catches it by name, fixes the prompt with format instructions, and tests the parser on its own.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define Rating: score (a number)
parser = a PydanticOutputParser for Rating
chain = prompt (no shape in it) | model | parser
try the chain on "pizza"
    print the exact error type
```

```
# structured_output_practice.py — Failure section
parser = PydanticOutputParser(pydantic_object=Rating)
prompt = ChatPromptTemplate.from_template("Rate {thing} from 1 to 10.")
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
chain = prompt | model | parser
```

What's missing: the imports, the `Rating` class, and the `try/except` call.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
BROKEN_TEMPLATE = "Rate {thing} from 1 to 10."
FIXED_TEMPLATE  = "Rate {thing} from 1 to 10.\n{format_instructions}"

function build_rating_chain(template) -> Runnable:
    prompt from the template
    if the template has {format_instructions}: fill it with .partial()
    return prompt | model | parser

function get_rating(chain, thing) -> Rating | None:
    try: return chain.invoke(...)
    except OutputParserException: return None
```

Run the broken chain, the fixed chain, and `parser.parse("I'd say eight")`, then compare against the Solution.

Difference between Basic and Intermediate: Basic shows the failure once. Intermediate catches it by name, returns `None` to the caller, and shows the real fix.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution